Knowledge distillation into a 17M encoder with 10 % of the labels

Ettin-encoder-400m → Ettin-encoder-17m on SST-2

Group project, Efficient Models course, ITMO 2026

Motivation: labels are scarce, the accurate model is slow

- Labeling a whole dataset is expensive; raw texts of the same domain are free
- With 10 % of SST-2 labels, the 400M teacher reaches 94.1 %, the 17M student only 84.3 %
- The student is 63 times faster on CPU
- Question: how much of the 100 %-label accuracy can the 17M student get from 10 % of the labels and a teacher?

	Teacher	Student
Parameters, M	395.8	16.9
CPU, batch of 32, ms	5,817	92
T4, sentences/s	314	3,243
Accuracy, 10 % labels	94.1	84.3
Accuracy, 100 % labels	95.9	89.5

SST-2 validation accuracy in %, mean of 3 seeds (teacher at 100 %: 1), CPU: 2 threads

Plan of work

1. Fine-tune the teacher on a random 1, 5, 10, 25, 100 % of the SST-2 train set
2. Train the student with cross-entropy (CE) on the same labels: the baseline
3. Train the student with knowledge distillation (KD) on the labeled part only, and on all 67,349 sentences
4. Replace the random 10 % by k-center selection
5. Measure parameters, CPU latency, T4 throughput
6. Vary the KD temperature and weight; repeat with 32M and 68M students; 3 teacher seeds at 10 %

Budget: 5 days, Kaggle T4 sessions of 12 hours; the 74 runs took 197 + 199 min in two sessions on one T4.

Knowledge distillation (Hinton et al., 2015)

The student matches the teacher's class probabilities softened by temperature T:

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}, \qquad \mathcal{L} = (1-\alpha)\,\mathrm{CE}(y, p^s_{T=1}) + \alpha\, T^2\, \mathrm{KL}\big(p^t_T \,\|\, p^s_T\big)$$

- Soft targets carry how similar the classes are for each example; a one-hot label does not
- Gradients of the soft term scale as $1/T^2$, so it is multiplied by T^2
- MNIST: a small net made 146 test errors alone, 74 when trained on the soft targets of a large net (67 errors)
- Speech, 3 % of the training set: frame accuracy 44.5 % with hard targets, 57.0 % with soft targets, 58.9 % with all data

`https://arxiv.org/abs/1503.02531`

Distillation through unlabeled task data

Tang et al., 2019: BERT $\rightarrow$ BiLSTM

- Transfer set: the train set plus sentences made by masking, part-of-speech-guided word replacement and n-gram sampling
- Mean squared error on logits, distillation term only ($\alpha = 0$ worked best)
- SST-2 test: BiLSTM 86.7 $\rightarrow$ 90.7 %; BERT-large 94.9 %

`https://arxiv.org/abs/1903.12136`

Turc et al., 2019: Well-Read Students

- Pre-trained Distillation: masked language model pre-training of the compact model, then KD on unlabeled transfer data
- Pre-training the student matters even with millions of transfer examples
- 6-layer student, SST-2 test: 91.8 %

`https://arxiv.org/abs/1908.08962`

Our `kdu` runs are Pre-trained Distillation with the SST-2 train sentences as the transfer set.

Other distilled encoders

- DistilBERT (Sanh et al., 2019): KD during pre-training; 40 % smaller than BERT-base, 60 % faster, 97 % of its GLUE score
- TinyBERT (Jiao et al., 2019): KD of embeddings, attention, hidden states and logits, general then task-specific, with data augmentation; 4 layers, 7.5 times smaller and 9.4 times faster than BERT-base, 96.8 % of its GLUE score
- Stanton et al., 2021: students often disagree with their teachers even when they could match them; the authors trace it mainly to optimization

We distill only logits: teacher and student have different widths and depths, and the loss stays one line.

`https://arxiv.org/abs/1910.01108`, `https://arxiv.org/abs/1909.10351`, `https://arxiv.org/abs/2106.05945`

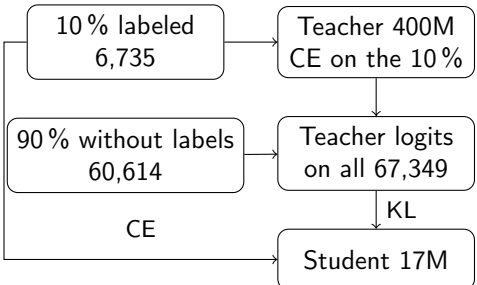
Models: Ettin encoders

- Ettin (Weller et al., 2025): paired encoders and decoders from 17M to 1B parameters, one recipe, one tokenizer, up to 2T tokens
- Architecture and recipe of ModernBERT (Warner et al., 2024): RoPE, alternating local and global attention, unpadding, 8,192-token context
- Teacher and student share the tokenizer: one tokenization for both, the teacher's logits are cached once
- 1B does not fit a T4 for AdamW fine-tuning in fp16

Ettin encoder	Non-embedding, M
400m, teacher	344.3
68m	42.6
32m	12.7
17m, student	4.0

`https://arxiv.org/abs/2507.11412`,
`https://arxiv.org/abs/2412.13663`

Setup



Run	Student texts	Loss
`hard_p<p>`	p % labeled	CE
`kd_p<p>`	p % labeled	CE + KL
`kdu_p<p>`	all 67,349	CE + KL

- `kdu`: CE on the labeled sentences, KL on all
- GLUE SST-2; the validation split (872) is the test set, no dev set, last step evaluated
- $T = 2$, $\alpha = 0.5$; AdamW, batch 32, fp16; teacher LR 3e-5, 3 epochs; student LR 1e-4, 5 epochs
- Students: 3 seeds, all from the seed-0 teacher; teachers: 1 seed, `teacher_p10`: 3

Frameworks and reproducibility

- PyTorch 2.11, Hugging Face Transformers 5.16 and Datasets; matplotlib for the figures
- uv with a lock file for local runs; Kaggle notebook for the T4
- Two scripts: `kd.py` (data, training, KD loss, k-center, benchmark, list of runs), `plots.py` (tables, figure)
- Each run writes `results/runs/<run>.json` with metrics and environment; a run whose file exists is skipped, so a new Kaggle session resumes from the output of the previous one
- Teacher logits and k-center indices are cached in `results/`
- `kd.py -smoke`: the whole pipeline on CPU in minutes

KD on unlabeled sentences closes 72 % of the gap at 10 % labels

Run	Accuracy, %
`teacher_p10`	94.1 ± 0.7
`hard_p10`	84.3 ± 1.3
`kd_p10`	84.1 ± 0.6
`kdu_p10`	88.1 ± 0.2
`hard_p100`	89.5 ± 0.6
`kd_p100`	89.0 ± 0.3

Mean ± std over 3 seeds

- `kdu_p10` gained 3.8 points over `hard_p10`, 72 % of the 5.2-point gap to `hard_p100`
- The gain came from the 90 % without labels: KD on the labeled 10 % alone was within the seed spread of CE
- With all labels, KD from a 95.9 % teacher did not raise the student
- Teacher seeds: 93.7, 94.8, 93.7 %; the students distill from the first one

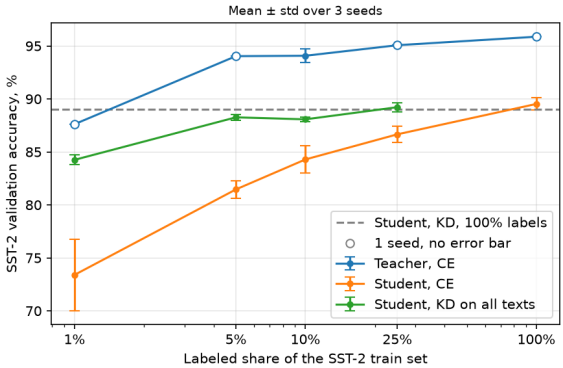
KL only with 10 % of the labels matched CE with all labels

Run	T	α	Accuracy, %
`kdu_p10_t1`	1	0.5	87.4 ± 0.3
`kdu_p10`	2	0.5	88.1 ± 0.2
`kdu_p10_t4`	4	0.5	88.8 ± 0.2
`kdu_p10_a1`	2	1	89.7 ± 0.3
`hard_p100`		0	89.5 ± 0.6

17M student, 10 % labels except `hard_p100`, 3 seeds

- KL only ($\alpha = 1$): +1.6 points over $\alpha = 0.5$, within the seed spread of CE on all 67,349 labels
- The student saw the gold labels only through a teacher trained on 10 % of them
- Each step of T from 1 to 2 to 4 added 0.7 points
- Tang et al. also found the distillation term alone best
- $T = 4$ with $\alpha = 1$ was not run

With KD, 5 % of the labels beat 25 % without it



Labels	`hard`, %	`kdu`, %
1 %	73.4 ± 3.4	84.3 ± 0.5
5 %	81.5 ± 0.8	88.3 ± 0.3
10 %	84.3 ± 1.3	88.1 ± 0.2
25 %	86.7 ± 0.8	89.2 ± 0.4
100 %	89.5 ± 0.6	

- KD with 1 % matched CE with 10 %
- The gain shrank from 10.9 points at 1 % to 2.6 at 25 %
- KD seed spread: 0.2–0.5, CE: 0.6–3.4

A larger student gains less from KD

Student	`hard_p10`	`kdu_p10`	`hard_p100`
Ettin 17M	84.3 ± 1.3	88.1 ± 0.2	89.5 ± 0.6
Ettin 32M	88.0 ± 1.0	90.2 ± 0.5	89.6 ± 0.9
Ettin 68M	89.6 ± 0.4	91.1 ± 0.4	92.4 ± 0.7

Accuracy in %, 3 seeds, $T = 2$, $\alpha = 0.5$, the same LR and epochs for all sizes

- KD gain at 10 % labels: 3.8, 2.2, 1.5 points
- CE with 10 % labels rose from 84.3 to 89.6 %
- 32M: KD with 10 % within the seed spread of CE with all labels
- 68M: KD closed 54 % of the gap
- CE with all labels: 17M and 32M both at 89.5–89.6 %

K-center selection of the 10 % lowered accuracy

- K-center greedy (Sener & Savarese, 2018): pick the point farthest from the chosen set until 10 %; a 2-approximation of the k-center problem
- Space: mean-pooled embeddings of the pretrained 400M teacher
- Two 2025 projects found selections with k-center or k-means better than random for CE on SST-2

`https://arxiv.org/abs/1708.00489`

Run	Random, %	K-center, %
`teacher`, seed 0	93.7	92.8
`hard`	84.3 ± 1.3	79.1 ± 1.0
`kdu`	88.1 ± 0.2	86.7 ± 0.3

- CE lost 5.2 points, KD 1.4
- Positive share: 57.3 % in the k-center subset, 55.2 % in the random one

Previous projects on SST-2 with 10 % of the labels

Project (ITMO 2025)	Model	100 %	Random 10 %	Best 10 %
ITMO-EDLM/data-efficiency	ModernBERT-base	94.7	75.8	92.1, entropy + k-center
llm-data-selection-bert	bert-tiny, 4.4M	81.9	74.7	79.2, quality + diversity
train-data-sampling	ModernBERT-base	95.1	92.1	92.9, k-means
This project, student	Ettin 17M	89.5	84.3	88.1, KD on all texts

- All three selected data and trained with CE; none used a teacher
- One run per config in each; the same ModernBERT-base with random 10 % got 75.8 and 92.1

The student is 63 times faster than the teacher on CPU

Model	Params, M	CPU, ms	T4, sent/s
Ettin-encoder-400m	395.8	5,817	314
Ettin-encoder-68m	68.4	736	1,278
Ettin-encoder-32m	32.0	237	2,423
Ettin-encoder-17m	16.9	92	3,243

CPU: median of 10 forward passes over 32 test sentences, 2 PyTorch threads, Kaggle; T4: fp16 over the 872 test sentences

- 23 times fewer parameters, 86 times fewer outside the embedding matrix
- 10.3 times the T4 throughput
- 68M is 8 times slower than 17M on CPU
- KD cost: one teacher pass over 67,349 sentences, cached

Conclusions and next steps

- With 10 % of SST-2 labels, KD through all train sentences gave the 17M student 89.7 % (KL only), against 84.3 % with CE and 89.5 % with all labels
- The gain came from the sentences without labels, not from soft targets on the labeled ones
- KD with 5 % of the labels beat CE with 25 %; KD runs varied less between seeds
- The KD gain shrank with student size, from 3.8 points at 17M to 1.5 at 68M
- K-center selection hurt both CE and KD students

Next steps:

- $T = 4$ with $\alpha = 1$, at every label share and student size
- Tune the student's LR and epochs on a dev split carved from the labeled 10 %
- Augmented or out-of-domain transfer sets; Docker image with pinned versions

`github.com/PE51K/itmo-ai-talent-hub-efficient-models-course/tree/main/project`